arXiv:2107.01272v5 [cs.LG] 3 Mar 2022

Physics-Guided Deep Learning for Dynamical Systems

Rui (Ray) Wang
Computer Science and Engineering
University of California, San Diego
ruw020@ucsd.edu

Rose Yu
Computer Science and Engineering
University of California, San Diego
roseyu@ucsd.edu

Abstract

Modeling complex physical dynamics is a fundamental task in science and engineering. Traditional physics-based models are sample efficient, interpretable but often rely on rigid assumptions. Furthermore, direct numerical approximation is usually computationally intensive, requiring significant computational resources and expertise. While deep learning (DL) provides novel alternatives for efficiently recognizing complex patterns and emulating nonlinear dynamics, its predictions do not necessarily obey the governing laws of physical systems, nor do they generalize well across different systems. Thus, the study of physics-guided DL emerged and has gained great progress. Physics-guided DL aims to take the best from both physics-based modeling and state-of-the-art DL models to better solve scientific problems. In this paper, we provide a structured overview of existing methodologies of integrating prior physical knowledge or physics-based modeling into DL, with a special emphasis on learning dynamical systems. We also discuss the fundamental challenges and emerging opportunities in the area.

1 Introduction

Modeling complex physical dynamics over a wide range of spatial and temporal scales is a fundamental task in a wide range of fields including, for example, fluid dynamics [134], cosmology [140], economics[32], and neuroscience [62].

Dynamical systems are mathematical objects that are used to describe the evolution of phenomena over time and space occurring in nature. Dynamical systems are commonly described with differential equations that are equations relate one or more unknown functions and their derivatives.

Definition 1. *Fix an integer $k \geq 1$ and let U denote an open subset of $\mathbb{R}^n$. Let $u : U \mapsto \mathbb{R}$ and we write $u(x) = u(x_1, ..., x_{n-1})$, where $x \in U$. Then an expression of the form*

$$\mathcal{F}(D^k u(x), D^{k-1} u(x), ..., Du(x), u(x), x) = 0 \tag{1}$$

is called a $k^{\text{-th}}$-order partial differential equation (or ordinary differential equation when $n = 1$), where $\mathcal{F} : \mathbb{R}^{n^k} \times \mathbb{R}^{n^{k-1}} \times ... \times \mathbb{R}^n \times \mathbb{R} \times U \mapsto \mathbb{R}$.

$\mathcal{F}$ models the dynamics of a n-dimensional state $x \in \mathbb{R}^n$ and it can be either a linear or non-linear operator. Since most dynamics evolve over time, one of the variables of u is usually time dimension. In general one must specify appropriate boundary and initial conditions of Equ.1 to ensure the existence/uniqueness of a solution. Learning dynamical systems is to search a good model $\mathcal{F}$ that can accurately describe the behavior of the physical process insofar as we are interested.

Physics as a discipline, has a long tradition of using first-principles to describe spatiotemporal dynamics. The laws of physics have greatly improved our understanding of the physical world. Many physics laws are described by systems of highly nonlinear differential equations that have

direct implications for understanding and predicting physical dynamics. However, these equations are usually too complicated to be solvable. The current paradigm of numerical methods for solution approximation is purely physics-based: known physical laws encoded in systems of coupled differential equations are solved over space and time via numerical differentiation and integration schemes [59; 60; 92; 63; 103; 120]. However, these methods are tremendously computationally intensive, requiring significant computational resources and expertise. An alternative is seeking simplified models that are based on certain assumptions and roughly can describe the dynamics, such as Reynolds-averaged Navier-stokes equations for turbulent flows and Euler equations for gas dynamics [22; 79; 138]. But it is highly nontrivial to obtain a simplified model that can describe a phenomenon to a satisfactory accuracy. More importantly, for many complex real-world phenomena, only partial knowledge of their dynamics is known. The equations may not fully represent the true system states.

Deep Learning (DL) provides efficient alternatives to learn high-dimensional spatiotemporal dynamics from massive datasets. It achieves so by directly predicting the input-output mapping and bypassing numerical integration. Recent works have shown that DL can generate realistic predictions and significantly accelerate the simulation of physical dynamics relative to numerical solvers, from turbulence modeling to weather prediction [144; 72; 69; 68; 70]. This opens up new opportunities at the intersection of DL and physical sciences, such as molecular dynamics[129; 131], epidemiology[156], cardiology[91; 143] and material science [93; 19].

Despite the tremendous progress, DL is purely data-driven by nature, which has many limitations. DL models still adhere to the fundamental rules of statistical inference. Without explicit constraints, DL models are prone to make physically implausible forecasts, violating the governing laws of physical systems. Additionally, DL models often struggle with generalization: models trained on one dataset cannot adapt properly to unseen scenarios with different distributions, known as distribution (covariate) shift. For dynamics learning, the distribution shift occurs not only because the dynamics are non-stationary and nonlinear, but also due to the changes in system parameters, such as initial and boundary conditions [146].

Neither DL alone nor purely physics-based approaches can be considered sufficient for learning complex dynamical systems in scientific domains. Therefore, there is a growing need for integrating traditional physics-based approaches with DL models so that we can make the best of both types of approaches. There is already a vast amount of work about physics-guided DL [154; 40; 16; 77; 68; 116; 17], but the focus on deep learning for dynamical systems is still nascent. Physics-guided DL offers a set of tools to blend these physical concepts such as differential equations and symmetry with deep neural networks. On one hand, these DL models offer great computational benefits over traditional numerical solvers. On the other hand, the physical constraints impose appropriate inductive biases on the DL models, leading to accurate simulation, scientifically valid predictions, reduced sample complexity, and guaranteed improvement in generalization to unknown environments.

This survey paper aims to provide a structured overview of existing methodologies of incorporating prior physical knowledge into DL models for learning dynamical systems. The paper is organized as below.

- Section 2 formulates the three main `learning problems` of physics-guided DL, including solving differential equations, dynamics forecasting, learning dynamics residuals.
- Section 3 describes the five `objectives` of physics-guided DL.
- Section 4$\sim$8 categorizes existing physics-guided DL approaches into four groups based on the way how physics and DL are combined. Each lead with a detailed review of our recent work as a case study and further categorized based on application, model architecture or learning problems.
  - Section 4: `Physics-guided loss function`: prior physics knowledge are imposed as additional soft constraints in the loss function.
  - Section 5: `Physics-guided architecture design`: physics knowledge are incorporated into the design of neural network modules.
  - Section 6: `Hybrid physics-DL models`: complete physics-based approaches are directly integrated into DL models.
  - Section 8: `Invariant and equivariant DL models`: DL models are designed to respect the symmetries for a given physical system.

- Section 9 summarizes the challenges in this field and discusses the emerging opportunities for future research.

2 Problem Formulation

In this paper, we mainly focus on the following three learning problems.

2.1 Solving Differential Equations

When $\mathcal{F}$ in Equ 1 is *known* but Equ 1 is too complicated to be solvable, researchers tend to directly solve the differential equations by approximating solution of $u(x)$ with a deep neural network, and enforcing the governing equations as soft constraint on the output of the neural nets during training at the same time[114; 113; 76]. This approach can be formulated as the following optimization problem,

$$\min_\theta \mathcal{L}(u) + \lambda_\mathcal{F} \mathcal{L}_\mathcal{F}(u) \tag{2}$$

$\mathcal{L}(u)$ denotes the misfit of neural nets predictions and the training data points. θ denotes the neural nets parameters. $\mathcal{L}_\mathcal{F}(u)$ is a constraint on the residual of the differential equation system under consideration and $\lambda_\mathcal{F}$ is a regularization parameter that controls the emphasis on this residual. The goal is then to train the neural nets to minimize the loss function in Equ 2.

2.2 Dynamics Forecasting

When $\mathcal{F}$ in Equ 1 is *unknown* or numerically solving Equ 1 require too much computation, many works studied learning high-dimensional spatiotemporal dynamics by directly predicting the input-output system state mapping and bypassing numerical discretization and integration [34; 68; 129; 147]. If we assume the first dimension x_1 of u in Equ 1 is the time dimension t, then the problem of dynamics forecasting can be defined as learning a map $f : \mathbb{R}^{n\times k} \mapsto \mathbb{R}^{n\times q}$ that maps a sequence of historic states to future states of the dynamical system,

$$f(u(t-k+1,\cdot),...,u(t,\cdot)) = u(t+1,\cdot),...,u(t+q,\cdot) \tag{3}$$

where k is the input length and q is the output length. f is commonly approximated with purely data-driven or physics-guided neural nets and the neural nets are optimized by minimizing the prediction errors of the state $\mathcal{L}(u)$.

2.3 Learning Dynamics Residuals

When $\mathcal{F}$ in Equ 1 is *partially know*, we can use neural nets to learn the errors or residuals made by physics-based models [33; 160; 66]. The key is to learn the bias of physics-based models and correct it with the help of deep learning. The final prediction of the state is composed of the simulation from the physics-based models and the residual prediction from neural nets as below,

$$\hat{u} = \hat{u}_\mathcal{F} + \hat{u}_{\text{NN}}. \tag{4}$$

where $\hat{u}_\mathcal{F}$ is the prediction obtained by numerically solving $\mathcal{F}$, $\hat{u}_{\text{NN}}$is the prediction from neural networks and $\hat{u}$ is the final prediction made by hybrid physics-DL models.

This learning problem generally involves two training strategy: 1) joint training: optimizing the parameters in the differential equations and the neural networks at the same time by minimizing the prediction errors of the system states. 2) two-stage training: we first fit differential equations on the training data and obtain the residuals, then directly optimize the neural nets on predicting the residuals.

3 Objectives of Physics-Guided Deep Learning

This subsection provides an overview of the objectives of physics-guided DL for learning dynamical systems. By incorporating physical principles, governing laws, and domain knowledge into DL models, the rapidly growing field of physics-guided DL seeks to (1) accelerate data simulation (2) build physically scientifically valid models (3) improve the generalizability of DL models (4) solve complicated partial differential equations. We discuss each of the four objectives in detail.

3.1 Accelerate Data Simulation.

Simulation is an important method of analyzing, optimizing and designing real-world processes, which is easily verified, communicated, and understood. It serves as the surrogate modeling and digital twin and provides valuable insights into complex physical systems. Traditional physics-based simulations often rely on running numerical methods: known physical laws encoded in systems of coupled differential equations are solved over space and time via numerical differentiation and integration schemes[59; 92; 63; 103; 120]. These methods require significant computational resources because the discretization step size is usually confined to be very small due to stability constraints when the dynamics is complex. Moreover, the performance of numerical methods can highly depend on the initial guesses of unknown parameters [61].

Recently, DL has demonstrated great success in the automation, acceleration, and streamlining of highly compute-intensive workflows for science [116; 138; 72]. Deep dynamics models can directly approximate high-dimensional spatiotemporal dynamics by directly forecasting the future states and bypassing numerical integration [147; 34; 123; 109; 123; 148]. These models are trained to make forward predictions given the historic frames as input with one or more steps supervision, and can roll out up to hundreds of steps during inference. DL models is usually faster than classic numerical solvers by one to two orders of magnitude. Because DL is able to take much larger space or time steps than classical solvers [109].

Furthermore, deep generative models, such as generative adversarial network (GAN), can generate instantaneous high-resolution flow fields that are statistically similar to those of direct numerical simulation [70; 158]. The computer graphics community has also investigated using DL to speed up numerical simulations for generating realistic animations of fluids such as water and smoke [69; 138; 153]. However, the community focuses more on the visual realism of the simulation rather than the physical characteristics.

3.2 Build Scientifically Valid Models.

Despite the tremendous progress of DL for science, e.g., atmospheric science [116], computational biology [2], material science [19], quantum chemistry [128], it remains a grand challenge to incorporate physical principles in a systematic manner to the design, training, and inference of such models. Purely data-driven DL models still adhere to the fundamental rules of statistical inference. Without explicit constraints, DL models are prone to make physically implausible predictions, violating the governing laws of physical systems.

Thus, to build trustworthy predictive models for science and engineering, we need to leverage known physical principles to guide DL models to learn the correct underlying dynamics instead of simply fitting the observed data. For instance, [67; 64; 157; 12] improve the physical and statistical consistency of DL models by explicitly regularising the loss function with physical constraints. Hybrid DL models, e.g., [95; 6; 23] integrate differential equations in DL for temporal dynamics forecasting and achieve promising performance. [90] and [44] studied tensor invariant neural networks that can learn the Reynolds stress tensor while preserving Galilean invariance. [144] proposed a hybrid model by marrying RANS-LES coupling method and custom-designed U-net. [52] and [30] design Hamiltonian and Lagrangian neural networks that respect conservation laws.

3.3 Improve the generalizability of DL models

DL models often struggle with generalization: models trained on one dataset cannot adapt properly to unseen scenarios with distributional shifts that may naturally occur in dynamical systems [75; 3; 146]. In addition, most current approaches are still trained to model a specific system, making it challenging to meet the needs of the scientific domain with heterogeneous environments. Thus, it is imperative to develop generalizable DL models that can learn and generalize well across systems with various parameter domains, initial and boundary conditions.

Prior physical knowledge can be considered as an inductive bias that can place a prior distribution on the model class and shrink the model parameter search space. With the guide of the inductive bias, DL models can better capture the underlying dynamics from the data that are consistent with physical laws. Across different data domains and systems, the laws of physics stay constant. Hence,

integrating physical laws in DL enables the models to generalize outside of the training domain and even to different systems.

Embedding symmetries into DL models is one way to improve the generalization, which we will discuss in detail in subsection 8. For example, [147] demonstrated that the encoding of rotation, scaling and uniform motion symmetries into DL models greatly improves the generalization on forecasting turbulence.

There are many other ways to improve the generalization of DL models by incorporating other physical knowledge. [148] proposed a model-based meta-learning approach that can generalize across heterogeneous domains. It contains a prediction network which learns the shared dynamics of the entire domain, and an encoder which infers the parameters of the task. [43] encodes Lyapunov stability into an autoencoder model for predicting fluid flow and sea surface temperature. They show improved generalizability and reduced prediction uncertainty for neural nets that preserve Lyapunov stability. [130] shows adding spectral normalization to DNN to regularize its Lipschitz continuity can greatly improve the generalization to new input domains on the task of drone landing control.

3.4 Solve Complicated Partial Differential Equations

Although the governing equations of many physical systems are known, finding approximate solutions using numerical algorithms and computers is still prohibitively expensive. DL models may greatly reduce the computation since some models, such as graph neural nets [4; 88], can be independent of resolution and can take much larger space and time steps during inference than classical solvers. For example, [96; 97] designed two *PDE-Net* that can accurately predict the dynamical behavior of data and have the potential to reveal the underlying PDE model that drives the observed data. The most common approach is that deep neural networks can directly approximate the solution of complex coupled differential equations via gradient-based optimization, which is the so-called physics-informed neural network (PINN) models. This approach has shown success in approximating a variety of PDEs [113; 112; 21; 54]. However, poor generalization to the unseen domain and slow convergence in training may have limited its applicability to complex physics problems [76].

3.5 Search for Physical laws

One of the main themes of science is the search for fundamental laws of practical problems [40]. When the governing equations of dynamical systems are known, they allow for accurate mathematical modeling, robust forecasting, and increased interpretability. However, dynamical systems in many fields, such as epidemiology, finance, and neuroscience, have no formal analytical description. Therefore, physics-guided DL can also assist the search for unknown physical laws.

One common paradigm for data-driven discovery of governing equations is to first define a large set of possible mathematical basis functions and then learn the coefficients. The seminal work by [18] proposed to find ordinary differential equations by creating a dictionary of possible basis functions and applying sparse regression to select the relevant basis. [80; 117] further extend this method by using neural networks to construct the dictionary of basis functions. [20] contributed to this trend by introducing an efficient first-order conditional gradient algorithm for solving the optimization problem of finding the best sparse fit to observational data in a large library of potential nonlinear models. However, the scalability, overfitting issues and over-reliance on high quality measurement data herein have been the critical concerns [115].

4 Physics-Guided Loss Functions and Regularization

Complex physical dynamics occur over a wide range of spatial and temporal scales. Standard DL models may simply fit the observed data while failing to learn the correct underlying dynamics, thus leading to low physical consistency and poor generalizability. One of the simplest and most widely used approaches to incorporate physical constraints is via designing loss functions (regularization). Physics-guided loss functions (regularization) can assist DL models to capture correct and generalizable dynamic patterns that are consistent with physical laws. Furthermore, the loss functions constrained by physics laws can reduce the possible search space of parameters. This approach is sometimes referred to as imposing differentiable "soft" constraints, which will be contrasted with imposing "hard" constraints (physics-guided architecture) in the next section. In this chapter, we

will start with a simple example of physics-guided loss from our previous work, and then categorize this type of methods based on their objectives, including improving prediction, accelerating data generaion and solving differential equations.

4.1 Case Study: Divergence Free Regularizer

One of our earlier works [145] studied the task of forecasting two-dimensional raw velocity fields of an incompressible turbulent flow, as shown in Figure 1. Incompressible fluid flows have zero divergence everywhere. The DL models are trained to make forward predictions given the historical velocity fields, analogous to video prediction. In the proposed physics-guided DL model, Turbulent Flow Net (`TF-Net`), apart from the regular mean square error(MSE) loss between the target and the prediction, it includes an additional divergence-free regularizer to reduce the divergence of turbulent flow predictions during training, as shown below.

$$Loss = MSE(\hat{\boldsymbol{w}}, \boldsymbol{w}) + \lambda \|\nabla \cdot \hat{\boldsymbol{w}}\|^2 \tag{5}$$

where $\boldsymbol{w}(\mathbf{x}, t)$ is the velocity field over space $\mathbf{x}$ and time t. ∇ is the vector differential operator. λ is a hyper-parameter that controls the strength of the divergence free regularizer. By penalizing the divergence of the predictions, the proposed model is able to obey the mass conservation law and generate predictions with almost zero divergence. Thus, generally speaking, a physics-guided loss function includes an additional physics-based regularizer that ensures consistency with physical laws and is controlled by a hyper-parameter.

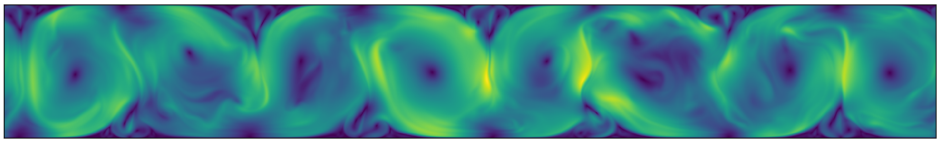


Figure 1: A snapshot of the velocity norm ($\sqrt{v_x^2 + v_y^2}$) fields of the 2D Rayleigh-Bénard convection flow [26]. The spatial resolution is 1792 x 256 pixels.

4.2 Improving Prediction Performance

Similar to the divergence-free loss in Eqn (5), physics-guided loss functions or regularization have shown great success in improving the prediction performance, especially the physical consistency of DL models. [67] used neural nets to model lake temperature at different times and different depths. They ensure that the predictions are physically meaningful by regularizing that the denser water predictions are at lower depths than predictions of less dense water. [64] further introduced a loss term that ensures thermal energy conservation between incoming and outgoing heat fluxes for modeling lake temperature. [13] designed conservation layers to strictly enforce conservation laws in their NN emulator of atmospheric convection. [11] introduced a more systematic way of enforcing nonlinear analytic constraints in neural networks via constraints in the loss function. [163] incorporated the loss of atomic force and atomic energy into neural nets for improved accuracy of simulating molecular dynamics. [93] proposed a novel multi-fidelity physics-constrained neural network for material modeling, in which the neural net was constrained by the losses caused by the violations of the model, initial conditions, and boundary conditions. [38] proposed a novel paradigm for spatiotemporal dynamics forecasting that performs spatiotemporal disentanglement using the functional variable separation. The specific-designed time invariance and regression loss functions ensure the separation of spatial and temporal information.

The Hamiltonian of a system is the sum of the kinetic energies of all particles, plus the potential energy of the particles associated with the system. [52] proposed *Hamiltonian Neural Nets (HNN)* that parameterizes a Hamiltonian with a neural network and then learn it directly from data. The conservation of desired quantities is constrained in the loss function during training. The proposed *HNN* has shown success in predicting mass-spring and pendulum systems. Lagrangian mechanics models the energies in a system rather than the forces. [30] proposed *Lagrangian Neural Nets (LNN)* used a neural network to parameterize the Lagrangian function that is the kinetic energy (energy of motion) minus the potential energy. They trained the neural network with the Euler-Lagrange

constraint loss functions such that it can learns to approximately conserve the total energy of the system. [47] further simplify the `HNN` and `LNN` via explicit constraints. [83] further introduced a meta-learning approach in `HNN` to find the structure of the Hamiltonian that can be adapted quickly to a new instance of a physical system. [166] benchmark recent energy-conserving neural network models based on Lagrangian/Hamiltonian dynamics on four different physical systems.

4.3 Data Generation

Simulation is an important method of analyzing, optimizing and designing real-world processes. Current numerical methods require significant computational resources when solving chaotic and complex differential equations. Because numerical discretization step size is confined to be very small due to stability constraints [61]. Also, the estimation of unknown parameters by fitting equations to the observed data requires much manual engineering in each application since the optimization of the unknown parameters in the system highly depends on the initial guesses. Thus, there is an increasing interest in utilizing deep generative models for simulating complex physical dynamics. Many works also imposed physical constraints in the loss function for better physical consistency.

For instance, [157] enforced the constraints of covariance into standard Generative Adversarial Networks (GAN) via statistical regularization, which leads to faster training and better physical consistency compared with standard GAN. [158] proposed *tempoGAN* for super-resolution fluid flow, in which an advection difference loss is used to enforce the temporal coherence of fluid simulation. [152] modified *ESRGAN*, which is a conditional GAN designed for super-resolution, by replacing the adversarial loss with a loss that penalizes errors in the energy spectrum between the generated images and the ground truth data. Conditional GAN is applied to emulating numeric hydroclimate models in [99]. The simulation performance is further improved by penalizing the snow water equivalent via loss function. [69] proposed a generative model to simulate fluid flows, in which a novel stream function-based loss function is designed to ensure divergence-free motion for incompressible flows. [50] proposed a physics-informed convolutional model for flow super-resolution, in which the physical consistency of the generated high-resolution flow fields is improved by minimizing the residuals of Navier-Stokes equations.

4.4 Solving Differential Equations

The partial differential equations (PDEs) for dynamical systems are often derived by starting from governing first principles. It is very difficult to find analytical solutions for many complex PDEs. Neural networks have been widely used to solve complex differential equations, where PDE constraints are enforced using the loss function [168; 111; 113].

[112; 114; 1; 132] directly approximate the solution differential equations with fully connected neural networks given space coordinates and time stamps as input, which is the so-called physics-informed neural network (PINN) model. They used automatic differentiation to differentiate the neural networks to calculate the first or second order derivatives with respect to their input coordinates and time. Then the governing equation can be enforced in the loss function as we formulate in the Section 2.1. PINNs have been demonstrated efficiency and accuracy in learning simple dynamics.

However, [76] pointed out that PINNs often fail to learn complex physical phenomena because the PDE regularizations make the optimization problem much more difficult. The authors also proposed two ways to alleviate this optimization problem. One is to start by training the PINN on a small constraint coefficient and then gradually increase the coefficient instead of using a big coefficient right away. The other one is training the PINN to predict the solution one time step at a time instead of the entire space-time at once.

4.5 Pros and Cons

As we can see, physics-guided loss functions are easy to design and use, and have the capability of improving the prediction accuracy and physical consistency. But they are usually considered as soft constraints because the physical constraints incorporated in the loss functions are not strictly enforced. In other words, the desired physical properties are not guaranteed via physics-guided loss function. Furthermore, PDE regularization may make loss landscapes more complex and cause optimization issues that are hard to address [76]. Additionally, there might be a trade-off between prediction

errors and physics-guided regularizers. For instance, in the case study, although constraining the the model with the divergence-free regularizer can reduce the divergence of the predictions, too much regularization has the side effect of smoothing out the small eddies in the turbulence, which may result in a larger prediction error [144].

5 Physics-Guided Design of Architecture

While incorporating physical constraints as regularizers in the loss function can improve performance, DL is still used as a black box model in most cases. The modularity of neural networks offers opportunities for the design of novel neurons, layers or blocks that encode specific physical properties. The advantage of physics-guided NN architectures is that they can impose "hard" constraints that are strictly enforced, compared to the "soft" constraints described in the previous section. The "soft" constraints are much easier to design than hard constraints, yet not required to be strictly satisfied. DL models with physics-guided architectures have theoretically guaranteed properties, and hence are more interpretable and generalizable. In this chapter, we will start with a case study of `Turbulent-Flow Net (TF-Net)` that unifies a popular Computational Fluid Dynamics (CFD) technique and a custom-designed U-net. We further categorize other related methods based on their architecture design.

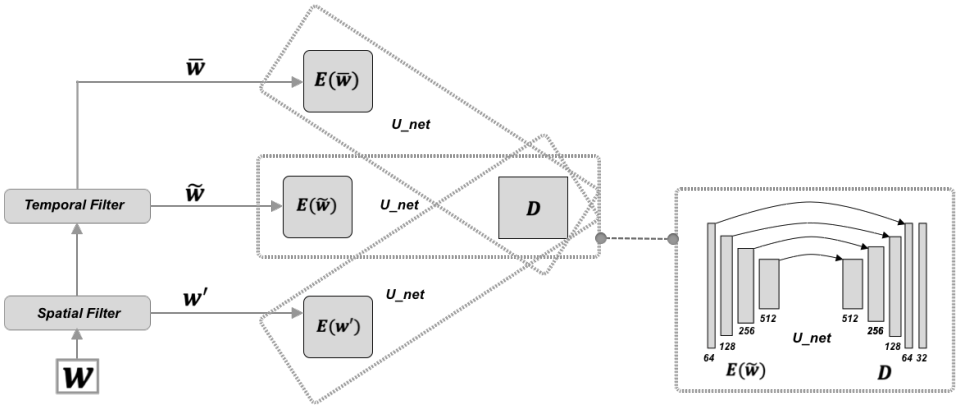


Figure 2: Turbulent Flow Net: three identical encoders to learn the transformations of the three components of different scales, and one shared decoder that learns the interactions among these three components to generate the predicted 2D velocity field at the next instant. Each encoder-decoder pair can be viewed as a U-net and the aggregation is weighted summation.

5.1 Case Study: Turbulent-Flow Net

`Turbulent-Flow Net (TF-Net)` [144] is a physics-guided DL model for turbulent flow prediction. As shown in Figure 2, it applies scale separation to model different ranges of scales of the turbulent flow individually. Computational fluid dynamics (CFD) techniques are at the core of present-day turbulence simulation. Direct Numerical Simulation (DNS) are accurate but not computationally feasible for practical applications. Great emphasis was placed on the alternative approaches including Large Eddy Simulation (LES) and Reynolds-averaged Navier-Stokes (RANS). Both resort to resolving large scales while modeling small scales, using various averaging techniques and/or low-pass filtering of the governing equations [103; 120].

One of the widely used CFD techniques, RANS-LES coupling approach [42], combines both Reynolds-averaged Equations (RANs) and Large Eddy Simulation (LES) approaches in order to take advantage of both methods. Inspired by RANS-LES coupling, `TF-Net` replaces `a priori` spectral filters with trainable convolutional layers. The turbulent flow is decomposed into three components, each of which is approximated by a specialized U-net to preserve the multiscale properties of the flow. A shared decoder learns the interactions among these three components and generate the final prediction. The motivation for this design is to explicitly guide the ML model to learn the nonlinear dynamics of large-scale and Subgrid-Scale Modeling motions as relevant to the task of spatiotemporal prediction. In other words, we need to force the model to learn not only the large eddies but also the

small ones. When we train a predictive model directly on the data with MSE loss, the model may overlook the small eddies and only focus on large eddies to achieve reasonably good accuracy.

Besides RMSE, physically relevant metrics including divergence and energy spectrum are used to evaluate the performance of the models' prediction. Figure 3 shows `TF-Net` consistently outperforms all baselines on physically relevant metrics (Divergence and Energy Spectrum) as well as average time to produce single velocity field. Constraining it with divergence free regularizer that we describe in the case study of last chapter can further reduce the RMSE and Divergence. Figure 4 shows the ground truth and predicted velocity along x direction by `TF-Net` and three best baselines. We see that the predictions by our TF-Net model are the closest to the target based on the shape and frequency of the motions. Thus, `TF-Net` is able to generate both accurate and physically meaningful predictions of the velocity fields that preserve critical quantities of relevance.

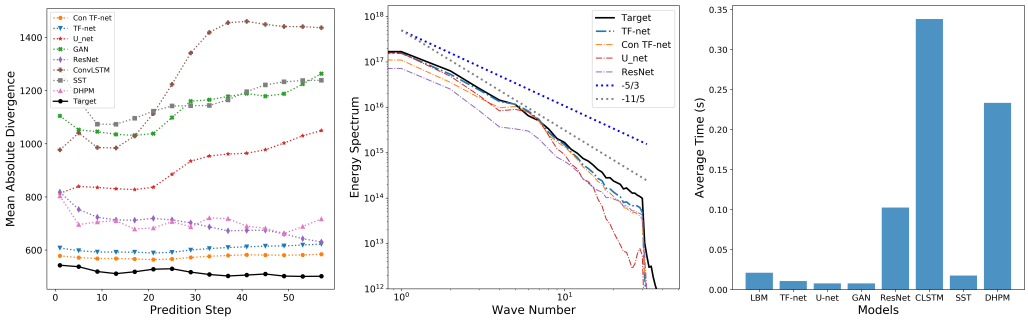


Figure 3: From left to right: Mean absolute divergence of different models' predictions at varying forecasting horizon; The Energy Spectrum of target, `TF-Net`, `U-net` and `ResNet` on the leftmost square sub-region; Average time to produce single 2D velocity field on single V100 GPU.

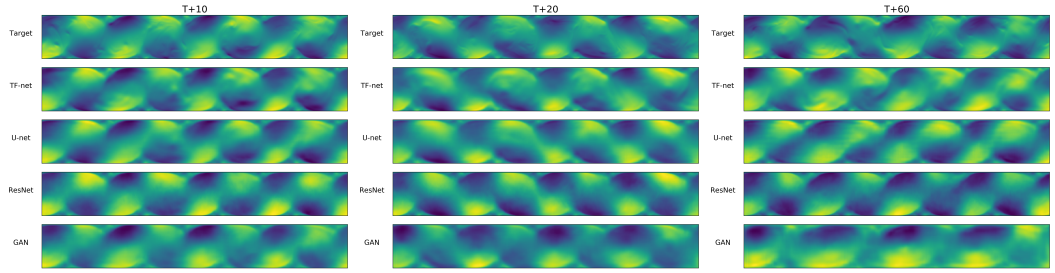


Figure 4: Ground truth and predicted velocity u by `TF-Net` and three best baselines (U-Net, ResNet and GAN) at time $T + 10$, $T + 30$ to $T + 60$ (suppose T is the time step of the last input frame).

5.2 Convolutional architecture

Convolutional architecture remains dominant in most tasks of computer vision, such as objection, image classification, and video prediction. Thanks to their efficiency and desired inductive biases, such as locality and translation equivariance, convolution neural nets have been widely applied to emulate and predict complex spatiotemporal physical dynamics. Researchers have proposed various ways to bake desired physical properties into the design of convolutional models.

For example, [65] proposed to enforce hard linear spatial PDE constraints within CNNs using the Fast Fourier Transform algorithm. [31] modified the LSTM units to introduce an intermediate variable to preserve monotonicity in a convolutional auto-encoder model for lake temperature. [104] proposed a physics-guided convolutional model, `PhyDNN`, which uses physics guided structural priors and physics-guided aggregate supervision for modeling the drag forces acting on each particle in a computational fluid dynamics-discrete element Method. [95] designed `HybridNet` for dynamics predictions that combines ConvLSTM for predicting external forces with model-driven computation with CeNN for system dynamics. `HybridNet` achieves higher accuracy on the tasks of forecasting heat convection-diffusion and fluid dynamics. [58] proposed to combine deep learning and a differentiable PDE solver

for understanding and controlling complex nonlinear physical systems over a long time horizon. [128] proposed continuous-filter convolutional layers for modeling quantum interactions. The convolutional kernel is parametrized by neural nets that take relative positions between any two points as input. They obtained a joint model for the total energy and interatomic forces that follows fundamental quantumchemical principles.

5.3 Graph Neural Networks

Standard convolutional neural nets only operate on the regular or uniform mesh such as images. Graph neural networks move beyond data on the regular grid towards modeling objects with arbitrary positions.

For instance, graph neural networks can model the Lagrangian representation of fluids and Euclidean representation on irregular meshes that CNNs cannot. [123] designed a deep encoder-processor-decoder graphic architecture for simulating fluid dynamics under Lagrangian description. The rich physical states are represented by graphs of interacting particles, and complex interactions are approximated by learned message-passing among nodes. [109] utilized the same architecture to learn mesh-based simulation. The authors directly construct graphs on the irregular meshes constructed in the numerical simulation methods. In addition, they proposed a adaptive remeshing algorithm that allows the model to accurately predict dynamics at both large and small scales. [15] further proposed two tricks to address the instability and error accumulation issues of training graph neural nets for solving PDEs. One is perturbing the input by a certain noise and only backpropagating errors on the last unroll step, and the other one is predicting multiple steps simultaneously in time. Both tricks make the model faster and more stable.

[4] proposed a *Neural Operator* approach that learns the mapping between function spaces, and is invariant to different approximations and grids. More specifically, it used the message passing graph network to learn the Green's function from the data and then the learned Green's function can used to compute the final solution of PDEs. [88] further extended it to *Fourier Neural Operator* by replacing the kernel integral operator with a convolution operator defined in Fourier space, which is much more efficient than *Neural Operator*. In [122], graph networks were also used to represent, learn, and infer robotics systems, bodies and joints. [87] proposed to learn compositional Koopman operators, using graph neural networks to encode the state into object-centric embeddings and using a block-wise linear transition matrix to regularize the shared structure across objects.

5.4 Koopman Theory

Koopman theory [74] shows that it is possible to represent a nonlinear dynamical system in terms of an infinite-dimensional linear Koopman operator acting on a Hilbert space of measurement functions of the system state. However, it is highly nontrivial to find the measurement functions that map the dynamics to the function space as well as an approximate and finite-dimensional Koopman operator. An approximation of the Koopman operator can be computed via the Dynamic Mode Decomposition algorithm [126] but we need to prepare nonlinear observables manually according to the underlying dynamics, which is not always possible since we usually do not have any prior knowledge about them. Thus, neural networks have been recently brought to learn the Koopman operator.

Many machine learning approaches hypothesize that there exists a data transformation that can be learnt by neural networks under which an approximate finite-dimensional Koopman operator is available. For instance, [159] and [135] proposed to use fully connected neural nets to directly map the observed dynamics to a dictionary of nonlinear observables that spans a Koopman invariant subspace. More specifically, this map is represented via an autoencoder network, embedding the observed dynamics onto a low-dimensional latent space, where the Koopman operator is approximated by a linear layer.[98] further generalize this idea for learning the Koopman operator to systems with continuous spectra. [7] also designed an autoencoder architecture based on Koopman theory to forecast physical processes. In the latent space, the consistency of both the forward and backward systems is ensured, while other models only consider the forward system. The proposed model performs well on systems that have both forward and backward dynamics for long time prediction.

5.5 Pros and Cons

Embedding physics into the design of the model architecture can enable physical principles strictly enforced and theoretically guaranteed. That leads to more interpretable and generalizable deep learning models. However, it is not trivial to design physics-guided architectures that perform and generalize well without hurting the representation power of neural nets. Hard inductive biases can greatly improve the sample efficiency of learning, but could potentially become restrictive when the size of the dataset is big enough for models to learn all the necessary inductive biases from the data.

6 Hybrid Physics-DL Model

Papers discussed in the previous two sections focus on incorporating the known properties of physical systems into the design of loss functions or neural network modules. In this section, we talk about works that directly combine pure physics-based models, such as numerical methods, with DL models.

6.1 Case Study: DeepGLEAM

Perhaps the simplest form of hybrid modeling is residual learning, where DL learns to predict the errors or residuals made by physics-based models. The key is to learn the bias of physics-based models and correct it with the help of DL models [49; 137].

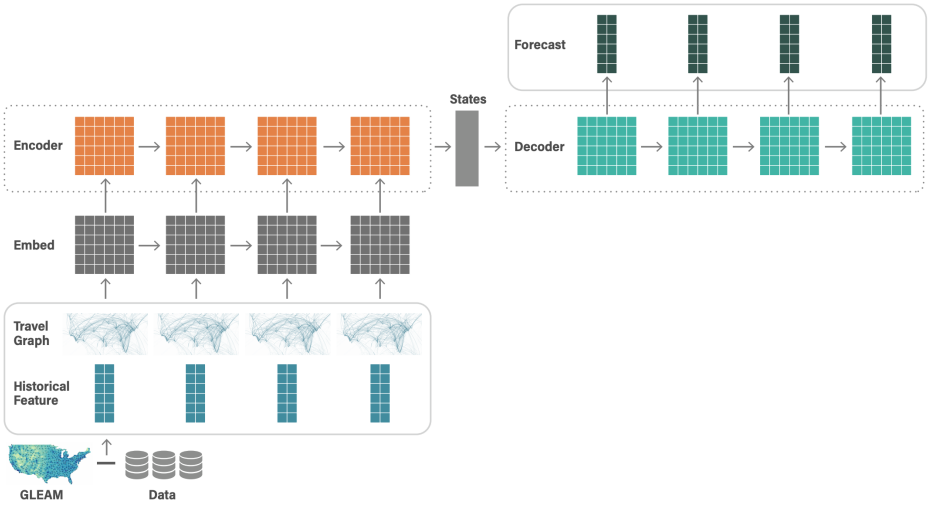


Figure 5: Architecture of *DeepGLEAM*: a DCRNN is trained with the residual between reported incident death and predictions from GLEAM.

A representative example is *DeepGLEAM* [156] for forecasting COVID-19 mortality, as shown in Figure 5. *DeepGLEAM* combines a mechanistic epidemic simulation model GLEAM with deep learning. It uses a Diffusion Convolutional RNN [86] (DCRNN) to learn the correction terms from GLEAM, which leads to improved performance. Figure 6 shows one week ahead COVID-19 death count predictions by GLEAM, DCRNN, and *DeepGLEAM*. We can see that *DeepGLEAM* can outperform purely mechanistic models and purely deep learning models.

6.2 Residual Modeling

Apart from *DeepGLEAM*, [33] combines graph neural nets with a CFD simulator run on coarse mesh to generate high-resolution fluid flow prediction. CNNs are used to correct the velocity field from the numerical solver on a coarse grid in [72]. [102] utilized neural networks for subgrid modelling of the LES simulation of two-dimensional turbulence. In [121], a neural network model is implemented in the reduced order modeling framework to compensate the errors from the model reduction. [66] proposed `DR-RNN` that is trained to find the residual minimizer of numerically discretized ODEs or

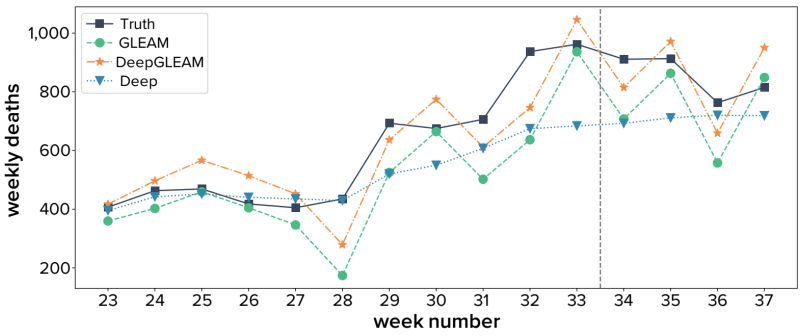


Figure 6: One week ahead COVID-19 death count forecasts visualization in California. Comparison shown for GLEAM, DeepGLEAM and DCRNN.

PDEs. They showed that `DR-RNN` can greatly reduce both computational cost and time discretization error of the reduced order modeling framework. [160] introduced the *APHYNITY* framework that can efficiently augment approximate physical models with deep data-driven networks. A key feature of their method is being able to decompose the problem in such a way that the data-driven model only models what cannot be captured by the physical model.

6.3 Intermediate Variable Modeling

DL models can be used to replace one or more components of physics-based models that are difficult to compute or unknown. For example, [138] replaced the numerical solver for solving Poisson's equations with convolution networks in the procedure of Eulerian fluid simulation, and the obtained results are realistic and showed good generalization properties. [107] proposed to use neural nets to reconstruct the model corrections in terms of variables that appear in the closure model. [35] applied a U-net to estimate the velocity field given the historical temperature frames, then used the estimated velocity to forecast the sea surface temperature based on the closed-form solution of the advection-diffusion equation. [100] combined the high-dimensional model representation that is represented as a sum of mode terms each of which is a sum of component functions with NNs to build multidimensional potential, in which NNs are used to represent the component functions that minimize the error mode term by mode term.

6.4 Neural Differential Equations

[24] developed a continuous depth NN for solving ordinary differential equations, *Neural ODE*. More specifically, they changed the traditionally discretized neuron layer depths into continuous equivalents such that the derivative of the hidden state can be parameterized using a neural network. The output of the network is computed using a blackbox differential equation solver. This allows for increased computational efficiency due to the simplification of the backpropagation step of training. *Neural ODE* offers tools to build continuous-time time series models, which can easily handle data coming at irregular intervals. It also has the advantage in building normalizing flow since it is easy to track the change in density even for unrestricted neural architecture

[39] introduced *Augmented Neural ODE* that is more expressive, empirically more stable and more lower computationally efficient than Neural ODEs. More importantly, it can learn the functions that have continuous trajectories mappings intersecting each other, which *Neural ODEs* cannot represent. [110] further extended this idea of continuous neural nets to graph convolutions, and proposed *Graph Neural ODE*. [94] proposed Neural Stochastic Differential Equation (*Neural SDE*), which models stochastic noise injection by stochastic differential equations. They demonstrated that incorporating the noise injection regularization mechanism to the continuous neural network can reduce overfitting and achieve lower generalization error. [91] proposed a Neural ODE-based generative time-series model that uses the known differential equation instead of treating it as hidden unit dynamics, so that they can integrate mechanistic knowledge into the Neural ODE.

7 Pros and Cons

Combining physics-based and deep learning models can enable leveraging both the flexibility of neural nets for modeling unknown parts of the dynamics and the interpretability and generalizbility of physics-based models. However, one potential downside of hybrid physics-DL models worth mentioning is that all or most of the dynamics could be captured by neural nets and the physics-based models contribute little to the learning. That would hurt the interpretability and the generalizablity of the model. We need to ensure optimal balance between the physics-based model and the deep learning model. We only need neural nets to model the information that cannot be represented by the physical prior.

8 Invariant and Equivariant DL Models

Symmetry has long been implicitly used in DL to design networks with known invariances and equivariances. Convolutional neural networks enabled breakthroughs in computer vision by leveraging translation equivariance [164; 82; 165]. Similarly, recurrent neural networks [118; 57], graph neural networks [125; 71], and capsule networks [119; 56] all impose symmetries. While the equivariant DL models have achieve remarkable success in image and text data [29; 150; 27; 25; 85; 73; 8; 155; 28; 46; 151; 37; 51; 133], the study of equivariant nets in learning dynamical systems has become increasingly popular recently. Since the symmetries can be integrated into neural nets through not only loss functions but also the design of neural net layers and there has been a large volume of works about equivariant and invariant DL models for physical dynamics, we discuss this topic separately in this section.

In physics, there is a deep connection between symmetries and physics. Noether's law gives a correspondence between conserved quantities and groups of symmetries. For instance, translation symmetry corresponds to the conservation of energy and rotation symmetry corresponds to the conservation of angular momentum. By building a neural network which inherently respects a given symmetry, we thus make conservation of the associated quantity more likely and consequently the model's prediction more physically accurate. Furthermore, by designing a model that is inherently equivariant to transformations of its inputs, we can guarantee that our model generalizes automatically across these transformations, making it robust to distributional shift.

A **group of symmetries** or simply **group** consists of a set G together with an associative composition map $\circ\colon G \times G \to G$. The composition map has an identity $1 \in G$ and composition with any element of G is required to be invertible. A group G has an **action** on a set S if there is an action map $\cdot\colon G \times S \to S$ which is compatible with the composition law. We say further that $\rho : G \mapsto GL(V)$ is a G**-representation** if the set V is a vector space and each group element $g \in G$ is represented by a linear map (matrix) $\rho(g)$ that acts on V. Formally, a function $f\colon X \to Y$ may be described as respecting the symmetry coming from a group G using the notion of equivariance.

Definition 2. *Assume a group representation ρ_{in} of G acts on X and ρ_{out} acts on Y. We say a function f is G-**equivariant** if*

$$f(\rho_{in}(g)(x)) = \rho_{out}(g) f(x) \tag{6}$$

*for all $x \in X$ and $g \in G$. The function f is G-**invariant** if $f(\rho_{in}(g)(x)) = f(x)$ for all $x \in X$ and $g \in G$. This is a special case of equivariance for the case $\rho_{\text{out}}(g) = 1$. See Figure 7 for an illustration of a rotation equivariant function.*

8.1 Case Study: Equivariant Deep Dynamics Models

[147] exploited the symmetries of fluid dynamics to design equivariant networks. The Navier-Stokes equations are invariant under the following five different transformations. Individually, each of these types of transformations generates a group of symmetries of the system.

- Space translation: $T^{\text{sp}}_{\boldsymbol{c}} \boldsymbol{w}(\boldsymbol{x}, t) = \boldsymbol{w}(\boldsymbol{x} - \boldsymbol{c}, t), \quad \boldsymbol{c} \in \mathbb{R}^2,$
- Time translation: $T^{\text{time}}_{\tau} \boldsymbol{w}(\boldsymbol{x}, t) = \boldsymbol{w}(\boldsymbol{x}, t - \tau), \quad \tau \in \mathbb{R},$
- Uniform motion: $T^{\text{um}}_{\boldsymbol{c}} \boldsymbol{w}(\boldsymbol{x}, t) = \boldsymbol{w}(\boldsymbol{x}, t) + \boldsymbol{c}, \quad \boldsymbol{c} \in \mathbb{R}^2,$
- Rotation/Reflection: $T^{\text{rot}}_{R} \boldsymbol{w}(\boldsymbol{x}, t) = R\boldsymbol{w}(R^{-1}\boldsymbol{x}, t), \; R \in O(2),$

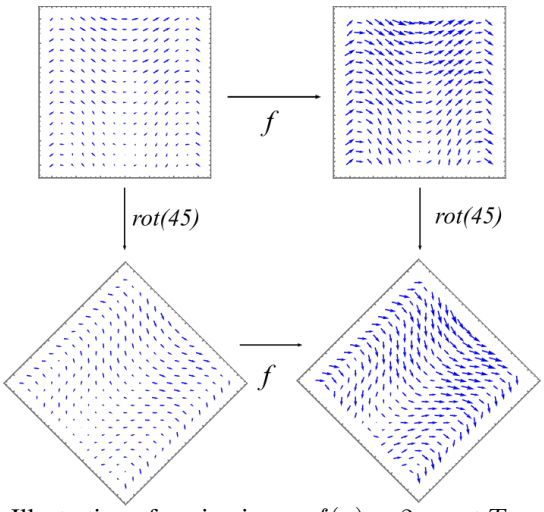


Figure 7: Illustration of equivariance: $f(x) = 2x$ w.r.t $T = \mathrm{rot}(\pi/4)$

- Scaling: $T_\lambda^{sc}\boldsymbol{w}(\boldsymbol{x}, t) = \lambda \boldsymbol{w}(\lambda \boldsymbol{x}, \lambda^2 t), \quad \lambda \in \mathbb{R}_{>0}$.

Consider a system of differential operators $\mathcal{D}$ acting on $\hat{\mathcal{F}}_V$. Denote the set of solutions $\mathrm{Sol}(\mathcal{D}) \subseteq \hat{\mathcal{F}}_V$. We say G is **a symmetry group of** $\mathcal{D}$ if G preserves $\mathrm{Sol}(\mathcal{D})$. That is, if φ is a solution of $\mathcal{D}$, then for all $g \in G$, $g(\varphi)$ is also. In order to forecast the evolution of a system $\mathcal{D}$, we model the forward prediction function f. Let $\boldsymbol{w} \in \mathrm{Sol}(\mathcal{D})$. The input to f is a collection of k snapshots at times $t-k, \ldots, t-1$ denoted $\boldsymbol{w}_{t-i} \in \mathcal{F}_d$. The prediction function $f\colon \mathcal{F}_d^k \to \mathcal{F}_d$ is defined $f(\boldsymbol{w}_{t-k}, \ldots, \boldsymbol{w}_{t-1}) = \boldsymbol{w}_t$. It predicts the solution at a time t based on the solution in the past. Let G be a symmetry group of $\mathcal{D}$. Then for $g \in G$, $g(\boldsymbol{w})$ is also a solution of $\mathcal{D}$. Thus $f(g\boldsymbol{w}_{t-k}, \ldots, g\boldsymbol{w}_{t-1}) = g\boldsymbol{w}_t$. Consequently, f is G-equivariant.

They tailored different methods for incorporating each symmetry into CNNs for spatiotemporal dynamics forecasting. CNNs are time translation-equivariant when used in an autoregressive manner. Convolutions are also naturally space translation equivariant. Scale equivariance in dynamics is unique as the physical law dictates the scaling of magnitude, space and time simultaneously. To achieve this, they replaced the standard convolution layers with group correlation layers over the group $G = (\mathbb{R}_{>0}, \cdot) \ltimes (\mathbb{R}^2, +)$ of both scaling and translations. The G-correlation upgrades this operation by both translating *and* scaling the kernel relative to the input,

$$\boldsymbol{v}(\boldsymbol{p}, s, \mu) = \sum_{\lambda \in \mathbb{R}_{>0}, t \in \mathbb{R}, \boldsymbol{q} \in \mathbb{Z}^2} \mu \boldsymbol{w}(\boldsymbol{p} + \mu \boldsymbol{q}, \mu^2 t, \lambda) K(\boldsymbol{q}, s, t, \lambda), \tag{7}$$

where s and t denote the indices of output and input channels. They add an axis to the tensors corresponding the scale factor μ.

The rotational symmetry was modeled using $\mathrm{SO}(2)$-equivariant convolutions and activations within the `E(2)-CNN` framework [150]. Uniform motion transformation is adding a constant vector field to the vector field, which is part of Galilean invariance and relevant to all non-relativistic physics modeling. And the uniform motion equivariance is enforced by conjugating the model with shifted input distribution. Basically, or each sliding local block in each convolutional layer, they shift the mean of input tensor to zero and shift the output back after convolution and activation function per sample. In other words, if the input is $\boldsymbol{\mathcal{P}}_{b \times d_{in} \times s \times s}$ and the output is $\boldsymbol{\mathcal{Q}}_{b \times d_{out}} = \sigma(\boldsymbol{\mathcal{P}} \cdot K)$ for one sliding local block, where b is batch size, d is number of channels, s is the kernel size, and K is the kernel, then

$$\boldsymbol{\mu}_i = \mathrm{Mean}_{jkl}\left(\boldsymbol{\mathcal{P}}_{ijkl}\right); \quad \boldsymbol{\mathcal{P}}_{ijkl} \mapsto \boldsymbol{\mathcal{P}}_{ijkl} - \boldsymbol{\mu}_i; \quad \boldsymbol{\mathcal{Q}}_{ij} \mapsto \boldsymbol{\mathcal{Q}}_{ij} + \boldsymbol{\mu}_i. \tag{8}$$

This will allow the convolution layer to be equivariant with respect to uniform motion. If the input is a vector field, this operation is applied to each element.

The DL models used are *ResNet* and *U-Net*, and their equivariant counterparts. Spatiotemporal prediction is done autoregressively. Standard RMSE and a RMSE computed on the energy spectra

are used to measure performance. The models are tested on Rayleigh-Bénard convection (RBC) and reanalysis ocean current velocity data. For RBC, the test sets have random transformations from the relevant symmetry groups applied to each sample. This mimics real-world data in which each sample has an unknown reference frame. For ocean data, tests are also performed on different time ranges and different domains from the training set, representing distributional shifts. Figure 8 shows the equivariant models perform significantly better than their non-equivariant counterparts on both simulated RBC data and real-world reanalysis ocean currents. They also show equivariant models also achieve much lower energy spectrum errors.

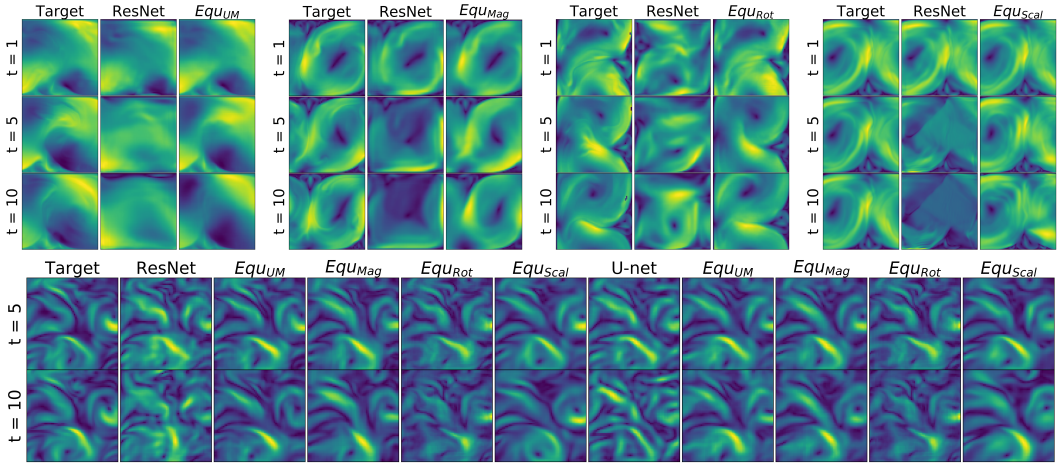


Figure 8: Top: The ground truth and the predicted velocity norm fields $\|\boldsymbol{w}\|_2$ of RBC at time step 1, 5 and 10 by the `ResNet` and four `Equ-ResNets` on four test samples applied with random uniform motion, magnitude, rotation and scaling transformations respectively. The first column is the target, the second is `ResNet` predictions, and the third is predictions by `Equ-ResNets`. Bottom: The ground truth and predicted velocity norm fields of ocean currents by `ResNet` (`Unet`) and four `Equ-ResNets` (`Equ-Unets`) on the test set.

8.2 Fluid Dynamics

We can exploit the symmetries in fluid dynamics to design more accurate and generalizable DL models. [148] utilized an encoder capable of extracting the time-invariant and translation-invariant part of a dynamical system, which then is used to guide the main forecaster to generate accurate predictions across heterogeneous domains. Time-invariance is achieved by using 3D convolution and time-shift invariant loss. [89] designed a *tensor basis neural network* that embeds the fundamental principle of rotational invariance for improved prediction of the metric tensor. It added a final higher-order multiplicative layer in NN to ensure the prediction of Reynolds stress anisotropy tensor lies on a rotationally invariant tensor basis. In [101], weights and biases of neurons are constrained so that predictions from the NN are guaranteed to preserve even/odd symmetry and energy conservation.

8.3 Molecular Dynamics

DL has also been used to accelerate computationally expensive molecular simulation [106]. Symmetry is ubiquitous in molecular physics. For instance, molecular conformations or coordinates have roto-translation equivariance.

[128] proposed *SchNet*, which is a continuous convolution framework that generalizes the CNN approach to continuous convolutions among to model particles at arbitrary positions. Continuous convolution kernels are generated by dense neural networks that operate on the interatomic distances, which ensures rotational and translation invariance of the energy. [5] designed *Cormorant*, a rotationally covariant neural network architecture for learning the behavior and properties of complex many-body physical systems. *Cormorant* achieves promising results in learning molecular potential energy surfaces on the MD-17 dataset and learning the geometric, energetic, electronic, and thermodynamic properties of molecules on the GDB-9 dataset.

[124] designed E(n)-equivariant graph neural network for predicting molecular properties. It updates edge features with the Euclidean distance between nodes and updates the coordinates of particles with the weighted sum of relative differences of all neighbors. [129] proposed to use score-based generative model for generating molecular conformation. The authors used equivariant graph neural networks to estimate the score function, which is the gradient fields of the log density of atomic coordinates, because it is roto-translation equivariant.

[131] proposed a model for autoregressive generation of 3D molecular structures with reinforcement learning (RL). The method uses equivariant state representations for autoregressive generation, built largely from *Cormorant*, and integrating such representations within an existing actor-critic RL generation framework. [142] computed the force by directly computing the derivatives of the predicted energy with respect to the coordinates to achieve the equivariance of the force. [162] proposed an end-to-end modeling framework that preserves all natural symmetries, including translation, rotation, and permutation, of a molecular system using an embedding procedure that maps the input to symmetry-preserving components.

Additionally, permutation invariance also exists in molecular dynamics. For instance, quantum mechanical energies are invariant if we exchange the labels of identical atoms. Hence, [10] ensures the energy permutation invariance by representing the total energy of the system as a sum of atomic contributions. [136] stated that enforcing equivariance to all permutations in graph neural nets can be very restrictive when modeling molecules. They proposed to decompose a graph into a collection of local graphs that are isomorphic to a pre-selected template graph, so that the sub-graphs can always be canonicalized to template graphs before convoluion applied. By doing this, the graph neural nets can not only be much more expressive but also locally equivariant.

8.4 Other Applications

[139] proposed to build equivariant neural networks based on the idea that nonlinear $O(d)$-equivariant functions can be universally expressed in terms of a lightweight collection of scalars, which are simpler to build. They demonstrated the efficiency and scalability of their proposed approach on two classical physics problems, calculating the total mechanical energy of particles and the total electromagnetic force, that obey all translation, rotation, reflection, boost and permutation symmetries.

In a traffic forecasting application, [141] proposed a novel model, `Equivariant Continuous COnvolution (ECCO)` that uses rotationally equivariant continuous convolutions to embed the symmetries of the system for improved trajectory prediction. The rotational equivariance is achieved by a weight sharing scheme within kernels in polar coordinates. `ECCO` achieves superior performance to baselines on two real-world trajectory prediction datasets, Argoverse and TrajNet++.

8.5 Pros and Cons

By designing a model that is inherently equivariant or invariant to transformations of its inputs, we can guarantee that our model generalizes automatically across these transformations, making it robust to distributional shift. Both empirically and theoretically, equivariant and invariant neural nets have demonstrated superior data and parameter efficiency compared to data augmentation techniques. More importantly, incorporating symmetries improves the physical consistency of neural nets because of the Noether's Law.

However, incorporating too many symmetries may overly constrain the representation power of neural nets and slow down both training and inference. In addition, many real world dynamics do not have perfect symmetries. A perfectly equivariant model that respects a given symmetry may have trouble learning partial or approximated symmetries in the real-world data. Thus, an ideal model for real world dynamics should be approximately equivariant and automatically learn the correct amount of symmetry in the data.

Recently, a few works explore the idea of building approximately equivariant models. [48] proposed the soft equivariant layer by directly summing up a flexible layer with one that has strong equivariance inductive biases to model the soft equivariance. [149] design approximately equivariant models by relaxing the weight sharing schemes in the equivariant convolutional layers. Both works demonstrate the benefits of approximately equivariant models in modeling real-world dynamics.

9 Discussion

In this paper, we systematically review the recent progress in physics-guided DL for learning dynamical systems. We discussed multiple ways to inject first-principle and physical constraints into DL including (1) physics-informed loss regularizers (2) physics-based models, (3) physics-guided design , and (4) symmetry. By integrating physical principles, the DL models can achieve better physical consistency, higher accuracy, increased data efficiency, improved generalization, and greater interpretability. Despite the great promise and exciting progress in the field, physics-guided AI is still at its infant stage. Below we review the emerging challenges and opportunities of learning physical dynamics with deep learning for future studies.

9.1 Improving Generalization

Generalization is a fundamental problem in machine learning. Unfortunately, most DL models for dynamics modeling are trained to model a specific system and still struggle with generalization. For instance, in turbulence modeling, DL models trained with a fixed boundary and initial conditions often fail to generalize to fluid flows with different characteristics. Incorporating prior physics knowledge can guide DL models to better learn complex patterns that are consistent with the physics laws from data, thus more generalizable to unseen scenarios. Another potential promising direction is meta-learning. For example, [148] proposed a model-based meta-learning method called *DyAd* which can generalize across heterogeneous domains of fluid dynamics. However, this model can generalize well on the dynamics with interpolated physical parameters but cannot extrapolate out of range of the physical parameters in the training set. A truly trustworthy and reliable model for learning physical dynamics should be able to extrapolate systems with various parameters, external forces, or boundary conditions while preserve high accuracy. Thus, further research into generalizable physics-guided DL is much needed.

9.2 Improving Robustness of Long-term Forecasting

Long-term forecasting of physical dynamics up to hundreds of steps ahead is another big challenge. Error accumulations and instability to the perturbation in the input are the two main issues that prevent neural networks being consistently accurate over a long forecasting horizon. One common training trick is adding noise to the input [15; 109], making models less sensitive to perturbations. [109; 9] utilized online batch normalization that is normalizing current training sample using running mean and standard deviation, which also increase the time horizon that the model can predict. These models are trained on a large amount of simulation data but for real-world problems, the real world data, such as experimental data of jet flow is very expensive to obtain. Thus, how to improve the robustness of prediction on limited training data is a big challenge.

9.3 Theoretical Analysis

The majority of the reviewed literature focuses on the methodological and practical aspects. The research into the theoretical analysis of learning non-stationary and chaotic dynamics with DL is lacking. Current learning theory of DL is based on the typical assumption of both training and test data being identically and independently distributed (i.i.d.) samples from some unknown distribution [161; 84; 105]. However, this assumption does not hold for most dynamical systems, where observations at different times and locations may be highly correlated. [146] empirically showed that DL models fail to generalize under shifted distributions in both the data and parameter domains that naturally happens in dynamical systems. [78] provided the first generalization guarantees for time series forecasting with sequence-to-sequence models. The derived upper bound is expressed in terms of measures of non-stationarity and correlation strength as well as the Rademacher complexity. To better understand the performance of DL on learning physical dynamics, we need to derive a generalization bound expressed in terms of the characteristics of the dynamics, such as the order and dimensions of the governing equations. Theoretical studies can also inspire research into model design and algorithm development for learning dynamical systems.

9.4 Control and Design

This survey primarily focuses on how DL can be used to model and predict complex physical dynamics given a fixed dataset. Next promising step would be, given the dynamics, how to design the environment to control it. For instance, automated computational fluid dynamics (CFD) analysis and control theory have been widely applied to aircraft design [41]. CFD can also be used to predict smoke and fire risks in buildings, quantify indoor environment quality and design natural ventilation systems [81]. How DL can assist and accelerate these processes still requires in-depth study.

9.5 Causal Inference in Dynamical Systems

A fundamental pursuit in science is to find causal relationships. In terms of dynamical systems, one may ask which variables influence other variables, either directly or indirectly through intermediates. While traditional approaches to discovery of causation are through conducting controlled real experiments [108; 14], data-driven approaches have been proposed to identify causal relations from observational data in the past few decades [55; 53]. Most data-driven approaches do not directly address learning causality with big data. Many questions remain open, such as using causality to improve DL models and disentangling complex and multiple treatments. Additionally, we are also interested in the system's response under interventions. For instance, when we use DL to model the climate dynamics, we need to make accurate predictions under different climate policy, such as carbon pricing policies and the development of clear energy, so that the government can make the correct decisions and better control the climate change.

9.6 Search for Physical Laws

Another promising direction is to seek physics laws with the help of DL. The search for fundamental laws of practical problems is the main theme of science. Once the governing equations of dynamical systems are found, they allow for accurate mathematical modeling, increased interpretability, and robust forecasting. However, current methods are limited to selecting from a large dictionary of possible mathematical terms [115; 127; 18; 80; 117]. The extremely large search space, limited high-quality experimental data, and the overfitting issues have been the critical concerns.

Another line of work is to discover symmetry from the observed data instead of the entire dynamics. For instance, [36] proposed Lie Algebra Convolutional Network, a novel architecture that can learn the Lie algebra basis and automatically discover symmetries from data. [167] factorized the weight matrix in a fully connected layer into a symmetry matrix and a vector of filter parameters. The two parts are learnt separately in the inner and outer loop training with the Model-Agnostic Meta-Learning algorithm (MAML) [45], which is an optimization-based meta-learning method, so that the symmetry matrix can learn the weight sharing pattern from the data. Still, research on data-driven methods based on DL for discovering physics laws are quite preliminary.

9.7 Efficient Computation

Given the rapid growth in high-performance computation, we need to improve automation, acceleration streamlining of highly compute-intensive workflows for science. We should focus on how to efficiently train, test, and deploy complex physics-guided DL models on large datasets and high performance computing systems, such that these models can be quickly utilized to solve real-world scientific problems. To really revolutionize the field, these DL tools need to become more scalable and transferable, and converge into a complete pipeline for the simulation and analysis of dynamical systems. A simple example is that we can integrate machine learning tools into the existing numerical simulation platforms, so that we do not need to move data between systems every time and we can easily use either or both types of methods for analyzing data.

In conclusion, given the availability of abundant data and rapid growth in computation, we envision that the integration of physics and DL will play an increasingly essential role in advancing scientific discovery and addressing important dynamics modeling problems.

References

[1] Ali Al-Aradi, Adolfo Correia, Danilo Naiff, Gabriel Jardim, and Yuri Saporito. Solving nonlinear and high-dimensional partial differential equations via deep learning. *arXiv preprint arXiv:1811.08782*, 2018.

[2] B. Alipanahi, Andrew Delong, Matthew T. Weirauch, and B. Frey. Predicting the sequence specificities of dna- and rna-binding proteins by deep learning. *Nature Biotechnology*, 33: 831–838, 2015.

[3] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in ai safety. *arXiv preprint arXiv:1606.06565*, 2019.

[4] Anima Anandkumar, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Nikola Kovachki, Zongyi Li, Burigede Liu, and Andrew Stuart. Neural operator: Graph kernel network for partial differential equations. In *ICLR 2020 Workshop on Integration of Deep Neural Models and Differential Equations*, 2020. URL `https://openreview.net/forum?id=fg2ZFmXFO3`.

[5] Brandon Anderson, Truong-Son Hy, and Risi Kondor. Cormorant: Covariant molecular neural networks. In *Advances in neural information processing systems (NeurIPS)*, 2019.

[6] Ibrahim Ayed, Emmanuel De Bézenac, Arthur Pajot, and Patrick Gallinari. Learning partially observed PDE dynamics with neural networks, 2019. URL `https://openreview.net/forum?id=HyefgnCqFm`.

[7] Omri Azencot, N. Erichson, Vanessa Lin, and Michael W. Mahoney. Forecasting sequential data using consistent koopman autoencoders. *arXiv Preprint arXiv:2003.02236*, 2020.

[8] Erkao Bao and Linqi Song. Equivariant neural networks and equivarification. *arXiv preprint arXiv:1906.07172*, 2019.

[9] Brian R. Bartoldson, Rui Wang, Brenda M. Ng, Phuoc Chanh N Nguyen, Jose E. Cadena Pico, Phan Nguyen, David P. Widemann, and USDOE National Nuclear Security Administration. Meshgraphnets, 2021. URL `https://www.osti.gov//servlets/purl/1834708`.

[10] J. Behler and M. Parrinello. Generalized neural-network representation of high-dimensional potential-energy surfaces. *Physical review letters*, 98 14:146401, 2007.

[11] Tom Beucler, M. Pritchard, S. Rasp, J. Ott, P. Baldi, and P. Gentine. Enforcing analytic constraints in neural-networks emulating physical systems. *arXiv: Computational Physics*, 2019.

[12] Tom Beucler, Michael Pritchard, Stephan Rasp, Pierre Gentine, Jordan Ott, and Pierre Baldi. Enforcing analytic constraints in neural-networks emulating physical systems. *arXiv preprint arXiv:1909.00912*, 2019.

[13] Tom Beucler, S. Rasp, M. Pritchard, and P. Gentine. Achieving conservation of energy in neural network emulators for climate modeling. *ArXiv*, abs/1906.06622, 2019.

[14] E. Bollt, J. Sun, and J. Runge. Introduction to focus issue: Causation inference and information flow in dynamical systems: Theory and applications. *Chaos*, 2018.

[15] Johannes Brandstetter, Daniel E. Worrall, and Max Welling. Message passing neural PDE solvers. In *International Conference on Learning Representations*, 2022. URL `https://openreview.net/forum?id=vSix3HPYKSU`.

[16] S. Brunton, B. R. Noack, and P. Koumoutsakos. Machine learning for fluid mechanics. *ArXiv*, abs/1905.11075, 2019.

[17] Steven L. Brunton. Applying machine learning to study fluid mechanics. *arXiv preprint arXiv:2110.02083*, 2021.

[18] Steven L. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Discovering governing equations from data: Sparse identification of nonlinear dynamical systems. *arXiv preprint arXiv:1509.03580*, 2015.

[19] Ruijin Cang, Hechao Li, Hope Yao, Yang Jiao, and Yi Ren. Improving direct physical properties prediction of heterogeneous materials from imaging data via convolutional neural network and a morphology-aware generative model. *arXiv: Computational Physics*, 2017.

[20] Alejandro Carderera, Sebastian Pokutta, Christof Schütte, and Martin Weiser. Cindy: Conditional gradient-based identification of non-linear dynamics – noise-robust recovery. *arXiv preprint arXiv:2101.02630*, 2021.

[21] G. Carleo and M. Troyer. Solving the quantum many-body problem with artificial neural networks. *Science*, 355:602 – 606, 2017.

[22] Bruno Chaoua. The state of the art of hybrid rans/les modeling for the simulation of turbulent flows. *Springer Netherlands*, 99:279–327, 2017. doi: https://doi.org/10.1007/s10494-017-9828-8.

[23] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems 31*, pages 6571–6583. Curran Associates, Inc., 2018. URL `http://papers.nips.cc/paper/7892-neural-ordinary-differential-equations.pdf`.

[24] Tian Qi Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. In *Advances in neural information processing systems*, pages 6571–6583, 2018.

[25] Benjamin Chidester, Minh N. Do, and Jian Ma. Rotation equivariance and invariance in convolutional neural networks. *arXiv preprint arXiv:1805.12301*, 2018.

[26] Dragos Bogdan Chirila. *Towards lattice Boltzmann models for climate sciences: The GeLB programming language with applications*. PhD thesis, University of Bremen, 2018.

[27] Taco S. Cohen and Max Welling. Group equivariant convolutional networks. In *International conference on machine learning (ICML)*, pages 2990–2999, 2016.

[28] Taco S. Cohen and Max Welling. Steerable CNNs. *arXiv preprint arXiv:1612.08498*, 2016.

[29] Taco S. Cohen, Maurice Weiler, Berkay Kicanaoglu, and Max Welling. Gauge equivariant convolutional networks and the icosahedral CNN. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97, pages 1321–1330, 2019.

[30] M. Cranmer, S. Greydanus, S. Hoyer, P. Battaglia, D. Spergel, and S. Ho. Lagrangian neural networks. *ArXiv*, abs/2003.04630, 2020.

[31] Arka Daw, R. Thomas, C. Carey, J. Read, A. Appling, and A. Karpatne. Physics-guided architecture (pga) of neural networks for quantifying uncertainty in lake temperature modeling. *ArXiv*, abs/1911.02682, 2020.

[32] Richard H. Day. Complex economic dynamics-vol. 1: An introduction to dynamical systems and market mechanisms. *MIT Press Books*, 1, 1994.

[33] Filipe de Avila Belbute-Peres, Thomas D. Economon, and J. Z. Kolter. Combining differentiable pde solvers and graph neural networks for fluid flow prediction. *ArXiv*, abs/2007.04439, 2020.

[34] Emmanuel de Bezenac, Arthur Pajot, and Patrick Gallinari. Deep learning for physical processes: Incorporating prior scientific knowledge. In *International Conference on Learning Representations*, 2018. URL `https://openreview.net/forum?id=By4HsfWAZ`.

[35] Emmanuel de Bezenac, Arthur Pajot, and Patrick Gallinari. Deep learning for physical processes: Incorporating prior scientific knowledge. In *International Conference on Learning Representations*, 2018. URL `https://openreview.net/forum?id=By4HsfWAZ`.

[36] Nima Dehmamy, Robin Walters, Yanchen Liu, Dashun Wang, and Rose Yu. Automatic symmetry discovery with lie algebra convolutional network. *Advances in Neural Information Processing Systems*, 34, 2021.

[37] Sander Dieleman, Jeffrey De Fauw, and Koray Kavukcuoglu. Exploiting cyclic symmetry in convolutional neural networks. In *International Conference on Machine Learning (ICML)*, 2016.

[38] Jérémie Donà, Jean-Yves Franceschi, sylvain lamprier, and patrick gallinari. {PDE}-driven spatiotemporal disentanglement. In *International Conference on Learning Representations*, 2021. URL `https://openreview.net/forum?id=vLaHRtHvfFp`.

[39] E. Dupont, A. Doucet, and Y. Teh. Augmented neural odes. In *NeurIPS*, 2019.

[40] Weinan E, Jiequn Han, and Linfeng Zhang. Integrating machine learning with physics-based modeling. *arXiv:2006.02619*, 2019.

[41] H. El-Sadi. Aircraft design analysis, cfd and manufacturing. *International Journal of Advanced Network, Monitoring and Controls*, 1:59–67, 2016.

[42] E.Labourasse and P.Sagaut. Reconstruction of turbulent fluctuations using a hybrid rans-les approach. *Journal of Computational Physics*, 182:301–336, 2002.

[43] N. Erichson, Michael Muehlebach, and Michael W. Mahoney. Physics-informed autoencoders for lyapunov-stable fluid flow prediction. *arXiv preprint arXiv:1905.10866*, 2019.

[44] Rui Fang, David Sondak, Pavlos Protopapas, and Sauro Succi. Deep learning for turbulent channel flow. *arXiv preprint arXiv:1812.02241*, 2018.

[45] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *International conference on machine learning*, pages 1126–1135. PMLR, 2017.

[46] Marc Finzi, Samuel Stanton, Pavel Izmailov, and Andrew Gordon Wilson. Generalizing convolutional neural networks for equivariance to lie groups on arbitrary continuous data. *arXiv preprint arXiv:2002.12880*, 2020.

[47] Marc Finzi, K. Wang, and A. Wilson. Simplifying hamiltonian and lagrangian neural networks via explicit constraints. *ArXiv*, abs/2010.13581, 2020.

[48] Marc Finzi, Gregory Benton, and Andrew G Wilson. Residual pathway priors for soft equivariance constraints. *Advances in Neural Information Processing Systems*, 34, 2021.

[49] U. Forssell and P. Lindskog. Combining semi-physical and neural network modeling: An example of its usefulness u. forssell and p. lindskog. 1997.

[50] Han Gao, Luning Sun, and J. Wang. Super-resolution and denoising of fluid flow using physics-informed convolutional neural networks without high-resolution labels. *arXiv: Fluid Dynamics*, 2020.

[51] Rohan Ghosh and Anupam K. Gupta. Scale steerable filters for locally scale-invariant convolutional neural networks. *arXiv preprint arXiv:1906.03861*, 2019.

[52] S. Greydanus, Misko Dzamba, and J. Yosinski. Hamiltonian neural networks. *ArXiv*, abs/1906.01563, 2019.

[53] Ruocheng Guo, Lu Cheng, Jundong Li, P. R. Hahn, and Huan Liu. A survey of learning causality with data. *ACM Computing Surveys (CSUR)*, 53:1 – 37, 2020.

[54] Jiequn Han, Linfeng Zhang, and E. Weinan. Solving many-electron schrödinger equation using deep neural networks. *J. Comput. Phys.*, 399, 2019.

[55] Michael Harradon, Jeff Druce, and Brian E. Ruttenberg. Causal learning and explanation of deep neural networks via autoencoded activations. *ArXiv*, abs/1802.00541, 2018.

[56] Geoffrey E Hinton, Alex Krizhevsky, and Sida D Wang. Transforming auto-encoders. In *International conference on artificial neural networks*, pages 44–51. Springer, 2011.

[57] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9 (8):1735–1780, 1997.

[58] Philipp Holl, Nils Thuerey, and Vladlen Koltun. Learning to control pdes with differentiable physics. In *International Conference on Learning Representations*, 2020. URL `https://openreview.net/forum?id=HyeSin4FPB`.

[59] B. Houska, F. Logist, M. Diehl, and J. Van Impe. A tutorial on numerical methods for state and parameter estimation in nonlinear dynamic systems. In D. Alberer, H. Hjalmarsson, and L. Del Re, editors, *Identification for Automotive Systems, Volume 418, Lecture Notes in Control and Information Sciences*, page 67–88. Springer, 2012.

[60] Thomas JR Hughes. *The finite element method: linear static and dynamic finite element analysis*. Courier Corporation, 2012.

[61] Arieh Iserles. *A first course in the numerical analysis of differential equations*. Number 44. Cambridge university press, 2009.

[62] Eugene M. Izhikevich. *Dynamical systems in neuroscience*. MIT press, 2007.

[63] J.C.Butcher. *Applied Numerical Mathematics*, volume 20. Elsevier B.V., 1996.

[64] Xiaowei Jia, Jared Willard, Anuj Karpatne, Jordan Read, Jacob Zwart, Michael Steinbach, and Vipin Kumar. Physics guided rnns for modeling dynamical systems: A case study in simulating lake temperature profiles. In *Proceedings of the 2019 SIAM International Conference on Data Mining*, pages 558–566. SIAM, 2019.

[65] Chiyu "Max" Jiang, Karthik Kashinath, Prabhat, and Philip Marcus. Enforcing physical constraints in {cnn}s through differentiable {pde} layer. In *ICLR 2020 Workshop on Integration of Deep Neural Models and Differential Equations*, 2020. URL `https://openreview.net/forum?id=q2noHUqMkK`.

[66] J. Kani and A. H. Elsheikh. Dr-rnn: A deep residual recurrent neural network for model reduction. *ArXiv*, abs/1709.00939, 2017.

[67] Anuj Karpatne, William Watkins, Jordan Read, and Vipin Kumar. Physics-guided neural networks (pgnn): An application in lake temperature modeling. *arXiv Preprint arXiv:1710.11431*, 2017.

[68] K. Kashinath, M. Mustafa, J-L. Wu A. Albert, C. Jiang, K. Azizzadenesheli S. Esmaeilzadeh, R. Wang, A. Chattopadhyay, A. Singh, A. Manepalli, D. Chirila, R. Yu, R. Walters, B. White, H. Xiao, H. A. Tchelepi, P. Marcus, A. Anandkumar, and P. Hassanzadeh. Physics-informed machine learning: case studies for weather and climate modelling. *Journal of Philosophical Transactions of the Royal Society A*, 2020.

[69] B. Kim, V. C. Azevedo, N. Thürey, Theodore Kim, M. Gross, and B. Solenthaler. Deep fluids: A generative network for parameterized fluid simulations. *Computer Graphics Forum*, 38, 2019.

[70] Junhyuk Kima and Changhoon Lee. Deep unsupervised learning of turbulence for inflow generation at various reynolds numbers. *arXiv:1908.10515v1*, 2019.

[71] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.

[72] D. Kochkov, J. A. Smith, A. Alieva, Qifeng Wang, M. Brenner, and Stephan Hoyer. Machine learning accelerated computational fluid dynamics. *ArXiv*, abs/2102.01010, 2021.

[73] Risi Kondor and Shubhendu Trivedi. On the generalization of equivariance and convolution in neural networks to the action of compact groups. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80, pages 2747–2755, 2018.

[74] B. O. Koopman. Hamiltonian systems and transformation in hilbert space. *Proceedings of the National Academy of Sciences of the United States of America*, 17 5:315–8, 1931.

[75] Wouter M. Kouw and Marco Loog. An introduction to domain adaptation and transfer learning. *arXiv preprint arXiv:1812.11806*, 2018.

[76] Aditi Krishnapriyan, Amir Gholami, Shandian Zhe, Robert Kirby, and Michael W Mahoney. Characterizing possible failure modes in physics-informed neural networks. *Advances in Neural Information Processing Systems*, 34, 2021.

[77] J. Kutz. Deep learning in fluid dynamics. *Journal of Fluid Mechanics*, 814:1–4, 2017.

[78] Vitaly Kuznetsov and Zelda E. Mariet. Foundations of sequence-to-sequence modeling for time series. *ArXiv*, abs/1805.03714, 2019.

[79] E. Labourasse and P. Sagaut. Advance in rans-les coupling, a review and an insight on the nlde approach. *Archives of Computational Methods in Engineering*, 11:199–256, 2004.

[80] John H. Lagergren, John T. Nardini, G. Michael Lavigne, E. Rutter, and K. Flores. Learning partial differential equations for biological transport models from noisy spatio-temporal data. *Proceedings of the Royal Society A*, 476, 2020.

[81] Tom Lawson. *Building Aerodynamics*. World Scientific Publishing Company, 2001. ISBN 9781860947537.

[82] Yann LeCun, Bernhard Boser, John S Denker, Donnie Henderson, Richard E Howard, Wayne Hubbard, and Lawrence D Jackel. Backpropagation applied to handwritten zip code recognition. *Neural computation*, 1(4):541–551, 1989.

[83] Seungjun Lee, Haesang Yang, and Woojae Seong. Identifying physical law of hamiltonian systems via meta-learning. In *International Conference on Learning Representations*, 2021. URL `https://openreview.net/forum?id=45NZvF1UHam`.

[84] P. Lemberger. On generalization and regularization in deep learning. *ArXiv*, abs/1704.01312, 2017.

[85] Karel Lenc and Andrea Vedaldi. Understanding image representations by measuring their equivariance and equivalence. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 991–999, 2015.

[86] Yaguang Li, Rose Yu, Cyrus Shahabi, and Yan Liu. Diffusion convolutional recurrent neural network: Data-driven traffic forecasting. In *International Conference on Learning Representations (ICLR '18)*, 2018.

[87] Yunzhu Li, Hao He, Jiajun Wu, D. Katabi, and A. Torralba. Learning compositional koopman operators for model-based control. *arXiv Preprint arXiv:1910.08264*, 2020.

[88] Zongyi Li, Nikola Borislavov Kovachki, Kamyar Azizzadenesheli, Burigede liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations*, 2021. URL `https://openreview.net/forum?id=c8P9NQVtmnO`.

[89] Julia Ling, Andrew Kurzawski, and Jeremy Templeton. Reynolds averaged turbulence modeling using deep neural networks with embedded invariance. *Journal of Fluid Mechanics*, 807: 155–166, 2016.

[90] Julia Ling, Andrew Kurzawskim, and Jeremy Templeton. Reynolds averaged turbulence modeling using deep neural networks with embedded invariance. *Journal of Fluid Mechanics*, 2017.

[91] Ori Linial, D. Eytan, and U. Shalit. Generative ode modeling with known unknowns. *Proceedings of the Conference on Health, Inference, and Learning*, 2021.

[92] Vadim Lisitsa, Galina Reshetova, and Vladimir Tcheverda. Finite-difference algorithm with local time-space grid refinement for simulation of waves. *Computational geosciences*, 16(1): 39–54, 2012.

[93] Dehao Liu and Yan Wang. Multi-fidelity physics-constrained neural network and its application in materials modeling. *Journal of Mechanical Design*, 141, 2019.

[94] Xuanqing Liu, Tesi Xiao, Si Si, Qin Cao, Sanjiv Kumar, and Cho-Jui Hsieh. Neural sde: Stabilizing neural ode networks with stochastic noise. *ArXiv*, abs/1906.02355, 2019.

[95] Yun Long, Xueyuan She, and Saibal Mukhopadhyay. Hybridnet: Integrating model-based and data-driven learning to predict evolution of dynamical systems. *ArXiv Preprint arXiv:1806.07439*, 2019.

[96] Zichao Long, Yiping Lu, Xianzhong Ma, and Bin Dong. Pde-net: Learning pdes from data. In *International Conference on Machine Learning*, pages 3214–3222, 2018.

[97] Zichao Long, Yiping Lu, and Bin Dong. Pde-net 2.0: Learning pdes from data with a numeric-symbolic hybrid deep network. *Journal of Computational Physics*, page 108925, 2019.

[98] Bethany Lusch, J. N. Kutz, and S. Brunton. Deep learning for universal linear embeddings of nonlinear dynamics. *Nature Communications*, 9, 2018.

[99] A. Manepalli and A. Albert. Emulating numeric hydroclimate models with physics-informed cgans. 2019.

[100] S. Manzhos and T. Carrington. A random-sampling high dimensional model representation neural network for building potential energy surfaces. *The Journal of chemical physics*, 125 8: 084109, 2006.

[101] Marios Mattheakis, Pavlos Protopapas, D. Sondak, Marco Di Giovanni, and Efthimios Kaxiras. Physical symmetries embedded in neural networks. *arXiv preprint arXiv:1904.08991*, 2019.

[102] R. Maulik, O. San, A. Rasheed, and P. Vedula. Subgrid modelling for two-dimensional turbulence using neural networks. *Journal of Fluid Mechanics*, 858:122–144, 2019.

[103] J. M. McDonough. *Introductory Lectures on Turbulence*. Mechanical Engineering Textbook Gallery, 2007. URL `https://uknowledge.uky.edu/me_textbooks/2`.

[104] Nikhil Muralidhar, Jie Bu, Z. Cao, Long He, N. Ramakrishnan, D. Tafti, and A. Karpatne. Phynet: Physics guided neural networks for particle drag force prediction in assembly. In *SDM*, 2020.

[105] Behnam Neyshabur, Srinadh Bhojanapalli, D. McAllester, and Nathan Srebro. Exploring generalization in deep learning. In *NIPS*, 2017.

[106] F. Noé, A. Tkatchenko, K. Müller, and C. Clementi. Machine learning for molecular simulation. *Annual review of physical chemistry*, 2020.

[107] Eric J. Parish and K. Duraisamy. A paradigm for data-driven predictive modeling using field inversion and machine learning. *J. Comput. Phys.*, 305:758–774, 2016.

[108] J. Pearl. Causal inference in statistics: An overview. *Statistics Surveys*, 3:96–146, 2009.

[109] Tobias Pfaff, Meire Fortunato, Alvaro Sanchez-Gonzalez, and Peter Battaglia. Learning mesh-based simulation with graph networks. In *International Conference on Learning Representations*, 2021. URL `https://openreview.net/forum?id=roNqYL0_XP`.

[110] Michael Poli, Stefano Massaroli, Junyoung Park, A. Yamashita, H. Asama, and Jinkyoo Park. Graph neural ordinary differential equations. *ArXiv*, abs/1911.07532, 2019.

[111] Christopher Rackauckas, Y. Ma, Julius Martensen, Collin Warner, K. Zubov, Rohit Supekar, D. Skinner, and Ali Ramadhan. Universal differential equations for scientific machine learning. *ArXiv*, abs/2001.04385, 2020.

[112] Maziar Raissi and George Em Karniadakis. Hidden physics models: Machine learning of nonlinear partial differential equations. *Journal of Computational Physics*, 357:125–141, 2018.

[113] Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. Physics informed deep learning (part i): Data-driven solutions of nonlinear partial differential equations. *arXiv preprint arXiv:1711.10561*, 2017.

[114] Maziar Raissi, Paris Perdikaris, and George E Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

[115] Chengping Rao, Pu Ren, Yang Liu, and Hao Sun. Discovering nonlinear PDEs from scarce data with physics-encoded learning. In *International Conference on Learning Representations*, 2022.

[116] Markus Reichstein, Gustau Camps-Valls, Bjorn Stevens, Martin Jung, Joachim Denzler, Nuno Carvalhais, and Prabhat. Deep learning and process understanding for data-driven earth system science. *Nature*, 566(7743):195–204, 2019. doi: 10.1038/s41586-019-0912-1. URL `https://doi.org/10.1038/s41586-019-0912-1`.

[117] Samuel H. Rudy, Steven L. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Data-driven discovery of partial differential equations. *arXiv preprint arXiv:1609.06401*, 2016.

[118] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *nature*, 323(6088):533–536, 1986.

[119] Sara Sabour, Nicholas Frosst, and Geoffrey E Hinton. Dynamic routing between capsules. *arXiv preprint arXiv:1710.09829*, 2017.

[120] Pierre Sagaut, Sebastien Deck, and Marc Terracol. *Multiscale and Multiresolution Approaches in Turbulence*. Imperial College Press, 2006.

[121] O. San and R. Maulik. Neural network closures for nonlinear model order reduction. *Advances in Computational Mathematics*, 44:1717–1750, 2018.

[122] Alvaro Sanchez-Gonzalez, N. Heess, Jost Tobias Springenberg, J. Merel, Martin A. Riedmiller, Raia Hadsell, and P. Battaglia. Graph networks as learnable physics engines for inference and control. *ArXiv*, abs/1806.01242, 2018.

[123] Alvaro Sanchez-Gonzalez, J. Godwin, T. Pfaff, Rex Ying, J. Leskovec, and P. Battaglia. Learning to simulate complex physics with graph networks. *ArXiv*, abs/2002.09405, 2020.

[124] Victor Garcia Satorras, Emiel Hoogeboom, and Max Welling. E (n) equivariant graph neural networks. In *International Conference on Machine Learning*, pages 9323–9332. PMLR, 2021.

[125] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. The graph neural network model. *IEEE transactions on neural networks*, 20(1):61–80, 2008.

[126] P. Schmid. Dynamic mode decomposition of numerical and experimental data. *Journal of Fluid Mechanics*, 656:5–28, 2008.

[127] Michael D. Schmidt and Hod Lipson. Distilling free-form natural laws from experimental data. *Science*, 324:81 – 85, 2009.

[128] Kristof Schütt, P. Kindermans, Huziel Enoc Sauceda Felix, Stefan Chmiela, A. Tkatchenko, and K. Müller. Schnet: A continuous-filter convolutional neural network for modeling quantum interactions. In *NIPS*, 2017.

[129] Chence Shi, Shitong Luo, and Minkai Xu1. Learning gradient fields for molecular conformation generation. *International Conference on Machine Learning*, 2021.

[130] Guanya Shi, Xichen Shi, Michael O'Connell, Rose Yu, Kamyar Azizzadenesheli, Animashree Anandkumar, Yisong Yue, and Soon-Jo Chung. Neural lander: Stable drone landing control using learned dynamics. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 9784–9790, 2019. doi: 10.1109/ICRA.2019.8794351.

[131] Gregor N. C. Simm, Robert Pinsler, Gábor Csányi, and José Miguel Hernández-Lobato. Symmetry-aware actor-critic for 3d molecular design. In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=jEYKjPE1xYN.

[132] Justin Sirignano and Konstantinos Spiliopoulos. Dgm: A deep learning algorithm for solving partial differential equations. *arXiv preprint arXiv:1708.07469*, 2018.

[133] Ivan Sosnovik, Michał Szmaja, and Arnold Smeulders. Scale-equivariant steerable networks. In *International Conference on Learning Representations*, 2020. URL https://openreview.net/forum?id=HJgpugrKPS.

[134] Steven H. Strogatz. *Nonlinear dynamics and chaos: with applications to physics, biology, chemistry, and engineering*. CRC press, 2018.

[135] Naoya Takeishi, Y. Kawahara, and T. Yairi. Learning koopman invariant subspaces for dynamic mode decomposition. In *NIPS*, 2017.

[136] Erik Thiede, Wenda Zhou, and Risi Kondor. Autobahn: Automorphism-based graph neural nets. *Advances in Neural Information Processing Systems*, 34, 2021.

[137] Michael L. Thompson and M. Kramer. Modeling chemical processes using prior knowledge and neural networks. *Aiche Journal*, 40:1328–1340, 1994.

[138] Jonathan Tompson, Kristofer Schlachter, Pablo Sprechmann, and Ken Perlin. Accelerating eulerian fluid simulation with convolutional networks. In *Proceedings of the 34th International Conference on Machine Learning-Volume 70*, pages 3424–3433. JMLR. org, 2017.

[139] Soledad Villar, David W Hogg, Kate Storey-Fisher, Weichi Yao, and Ben Blum-Smith. Scalars are universal: Equivariant machine learning, structured like classical physics. In *Advances in Neural Information Processing Systems*, 2021.

[140] John Wainwright. *Dynamical systems in cosmology*. Cambridge University Press, 2005.

[141] Robin Walters, Jinxi Li, and Rose Yu. Trajectory prediction using equivariant continuous convolution. In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=J8_GttYLFgr.

[142] Jiang Wang, C. Wehmeyer, F. Noé, and C. Clementi. Machine learning of coarse-grained molecular dynamics force fields. *ACS Central Science*, 5:755 – 767, 2019.

[143] Rui Wang, E. Huang, Uma Chandrasekaran, and Rose Yu. Aortic pressure forecasting with deep learning. *2020 Computing in Cardiology*, pages 1–4, 2020.

[144] Rui Wang, Karthik Kashinath, Mustafa Mustafa, Adrian Albert, and Rose Yu. Towards physics-informed deep learning for turbulent flow prediction. *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2020.

[145] Rui Wang, Karthik Kashinath, Mustafa Mustafa, Adrian Albert, and Rose Yu. Towards physics-informed deep learning for turbulent flow prediction. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1457–1466, 2020.

[146] Rui Wang, Danielle Maddix, Christos Faloutsos, Yuyang Wang, and Rose Yu. Bridging physics-based and data-driven modeling for learning dynamical systems. *arXiv Preprint arXiv:2011.10616*, 2020.

[147] Rui Wang, Robin Walters, and Rose Yu. Incorporating symmetry into deep dynamics models for improved generalization. 2021. URL https://openreview.net/forum?id=wta_8Hx2KD.

[148] Rui Wang, Robin Walters, and Rose Yu. Bridging physics-based and data-driven modeling for learning dynamical systems. *arXiv preprint arXiv:2102.10271*, 2021.

[149] Rui Wang, Robin Walters, and Rose Yu. Approximately equivariant networks for imperfectly symmetric dynamics. *arXiv preprint arXiv:2201.11969*, 2022.

[150] Maurice Weiler and Gabriele Cesa. General E(2)-equivariant steerable CNNs. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 14334–14345, 2019.

[151] Maurice Weiler, Fred A. Hamprecht, and Martin Storath. Learning steerable filters for rotation equivariant CNNs. *Computer Vision and Pattern Recognition (CVPR)*, 2018.

[152] B. White, A. Singh, and A. Albert. Downscaling numerical weather models with gans. 2019.

[153] Steffen Wiewel, Moritz Becher, and Nils Thuerey. Latent space physics: Towards learning the temporal evolution of fluid flow. In *Computer Graphics Forum*, volume 38, pages 71–82, 2019.

[154] Jared Willard, Xiaowei Jia, Shaoming Xu, Michael S. Steinbach, and Vipin Kumar. Integrating physics-based modeling with machine learning: A survey. *CoRR*, abs/2003.04919, 2020. URL `https://arxiv.org/abs/2003.04919`.

[155] Daniel E Worrall, Stephan J Garbin, Daniyar Turmukhambetov, and Gabriel J Brostow. Harmonic networks: Deep translation and rotation equivariance. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 5028–5037, 2017.

[156] Dongxian Wu, Liyao Gao, X. Xiong, Matteo Chinazzi, Alessandro Vespignani, Y. Ma, and Rose Yu. Deepgleam: a hybrid mechanistic and deep learning model for covid-19 forecasting. *ArXiv*, abs/2102.06684, 2021.

[157] Jin-Long Wu, Karthik Kashinath, Adrian Albert, Dragos Chirila, Prabhat, and Heng Xiao. Enforcing Statistical Constraints in Generative Adversarial Networks for Modeling Chaotic Dynamical Systems. *arXiv e-prints*, May 2019.

[158] You Xie, Erik Franz, Mengyu Chu, and Nils Thuerey. tempogan: A temporally coherent, volumetric gan for super-resolution fluid flow. *ACM Transactions on Graphics (TOG)*, 37(4): 95, 2018.

[159] E. Yeung, Soumya Kundu, and Nathan Oken Hodas. Learning deep neural network representations for koopman operators of nonlinear dynamical systems. *2019 American Control Conference (ACC)*, pages 4832–4839, 2019.

[160] Yuan Yin, Vincent LE GUEN, Jérémie DONA, Emmanuel de Bezenac, Ibrahim Ayed, Nicolas THOME, and patrick gallinari. Augmenting physical models with deep networks for complex dynamics forecasting. In *International Conference on Learning Representations*, 2021. URL `https://openreview.net/forum?id=kmG8vRXTFv`.

[161] C. Zhang, S. Bengio, Moritz Hardt, B. Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. *ArXiv*, abs/1611.03530, 2017.

[162] L. Zhang, Jiequn Han, Han Wang, W. Saidi, R. Car, and E. Weinan. End-to-end symmetry preserving inter-atomic potential energy model for finite and extended systems. In *NeurIPS*, 2018.

[163] Linfeng Zhang, Jiequn Han, Han Wang, R. Car, and E. Weinan. Deep potential molecular dynamics: a scalable model with the accuracy of quantum mechanics. *Physical review letters*, page 143001, 2018.

[164] Wei Zhang. Shift-invariant pattern recognition neural network and its optical architecture. In *Proceedings of annual conference of the Japan Society of Applied Physics*, 1988.

[165] Wei Zhang, Kazuyoshi Itoh, Jun Tanida, and Yoshiki Ichioka. Parallel distributed processing model with local space-invariant interconnections and its optical architecture. *Applied optics*, 29(32):4790–4797, 1990.

[166] Yaofeng Zhong, Biswadip Dey, and Amit Chakraborty. Benchmarking energy-conserving neural networks for learning dynamics from data. *arXiv Preprint arXiv:2012.02334*, 2020.

[167] Allan Zhou, Tom Knowles, and Chelsea Finn. Meta-learning symmetries by reparameterization. *arXiv preprint arXiv:2007.02933*, 2020.

[168] Yinhao Zhu, N. Zabaras, P. Koutsourelakis, and P. Perdikaris. Physics-constrained deep learning for high-dimensional surrogate modeling and uncertainty quantification without labeled data. *ArXiv*, abs/1901.06314, 2019.